THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Patiala, Punjab

COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

SMART AIRPORT MANAGEMENT SYSTEM

(SAMS)

Mid-Semester Evaluation

Submitted By

1024240048 Ridhi Mahajan
1024240053 Priyanka
1024240057 Diya Saini
1024240066 Ishika Jain

Group 5

Faculty
Anushka Chaudary

BE Third Year, Computer Science and Engineering

9 September

Contents

List of Figures

List of Tables

1. Project Selection Phase

1.1 Software Bid with Higher-Order Goal

The Smart Airport Management System (SAMS) is a centralized software platform designed to organize and manage major airport-related operational information through a single web-based interface.

The system focuses on improving information accessibility, reducing fragmentation between operational categories, and providing an integrated management environment for airport activities.

The higher-order goal of SAMS is to provide a structured digital platform through which authorized users can access and manage airport information efficiently.

Time-to-Value

SAMS is designed as a modular web application so that individual airport management functions can be accessed without requiring separate standalone systems for each category.

The centralized interface allows the administrator to view and manage operational information from one application, thereby reducing unnecessary switching between different interfaces.

Problem Statement

Airport operations involve several categories of information, including flight details, luggage records, visitor information, staff information, and transportation services.

Managing these categories separately can make information difficult to access and maintain. A centralized management platform is therefore required to organize this information in a consistent and accessible manner.

Proposed Solution

SAMS provides a centralized web-based management system consisting of:

- Secure user authentication.
- Role-based API authorization.
- Flight information management.
- Luggage information management.
- Visitor management.
- Staff management.
- Transportation management.
- Dashboard-based operational overview.
- Reporting and system information views.

The current prototype demonstration is primarily administrator-centric, while the back-end architecture supports additional user roles.

1.2 Technology Stack

The project uses a modern web application architecture consisting of a React frontend, an Express.js backend, and a MongoDB database.

Table I: Technology Stack

Technology	**Purpose**
React.js	Frontend user interface and component-based development
Vite	Frontend development and build tooling
React Router	Client-side routing and protected navigation
Axios	Communication between frontend and backend APIs
Node.js	Backend JavaScript runtime
Express.js	REST API and backend server framework
MongoDB	Database for persistent application data
Mongoose	Object Data Modeling for MongoDB
JWT	Authentication token generation and verification
bcryptjs	Password hashing and credential verification
Tailwind CSS	Frontend styling and responsive interface design
Recharts	Dashboard and report visualizations
Git/GitHub	Version control and project collaboration

1.3 Evaluation Criteria

The system is evaluated according to the following criteria:

- Correctness of authentication and protected API access.
- Functionality of major CRUD operations.
- Correct communication between frontend and backend.
- Database integration.
- Role-based access control at the API level.
- Usability of the centralized interface.
- Maintainability and modularity of the implementation.

1.4 Scalability

The modular architecture of SAMS allows additional airport management modules and role-specific interfaces to be incorporated without redesigning the entire system.

The use of REST APIs and separate frontend and backend layers also allows the system to be extended independently.

1.5 Risks and Mitigations

- **Unauthorized API access:** Protected routes use JWT authentication and role-based authorization middleware.
- **Invalid database data:** Mongoose schemas define validation rules and data types.
- **Frontend-backend inconsistency:** Axios provides a centralized API communication layer.
- **Incomplete role-specific interfaces:** The current prototype focuses on the administrator workflow, while additional interfaces can be implemented in future iterations.

1.6 Summary

SAMS provides a centralized software solution for managing airport-related operational information. The project combines a React frontend, Express.js backend, MongoDB database, JWT authentication, and role-based authorization into a modular web application.

1.7 Project Overview

The system follows a centralized workflow in which users interact with the React frontend, requests are sent through the Axios communication layer, and the Express.js backend processes authenticated API requests.

The backend communicates with MongoDB through Mongoose models.

Primary User

The Administrator is the primary authenticated user demonstrated in the current prototype.

Additional Roles

The backend defines roles for Staff, Airline Staff, Passenger, Visitor, and Transportation Operator, and role-based authorization is implemented at the API level.

However, the current prototype demonstration is administrator-centric and does not provide separate dedicated frontend workflows for each of these roles. Dedicated role-specific interfaces can be developed as future extensions.

Managed Categories

The system manages the following major categories:

- Users and authentication
- Flights
- Luggage
- Visitors
- Staff
- Transportation

System Components

The main components of SAMS are:

- Authentication context
- Protected frontend routes
- Axios API communication layer
- Express REST API
- Authentication and authorization middleware
- Controllers
- Mongoose models
- MongoDB database

2. Analysis Phase

2.1 Use-Case Analysis

Figure 1 represents the major actors and operations defined for SAMS.

The Administrator is the primary user demonstrated in the current prototype. The backend also defines additional roles including Staff, Airline Staff, Passenger, Visitor, and Transportation Operator, with role-based API permissions implemented for extensibility.

The current frontend demonstration focuses primarily on the centralized administrator workflow, while separate role-specific interfaces can be developed in future iterations.

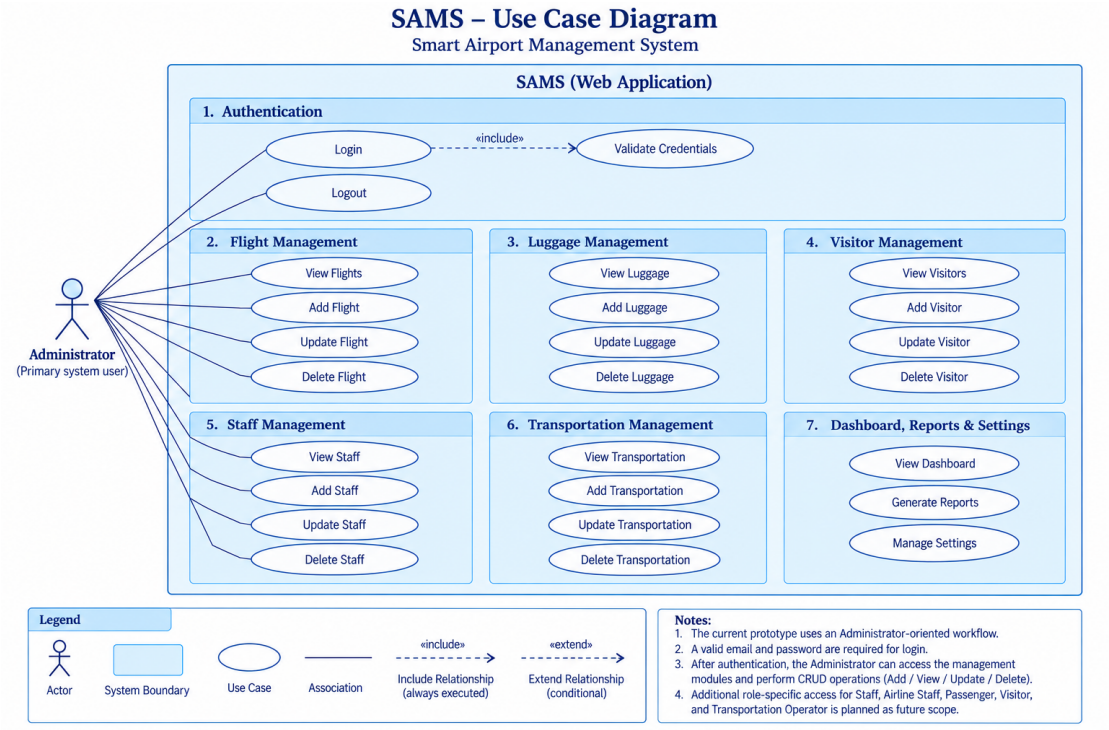

Figure 1: Use-Case Diagram

2.2 Activity Diagram

The activity diagram represents the major flow of interaction between the user, React frontend, Express.js backend, authentication and authorization middleware, and MongoDB database.

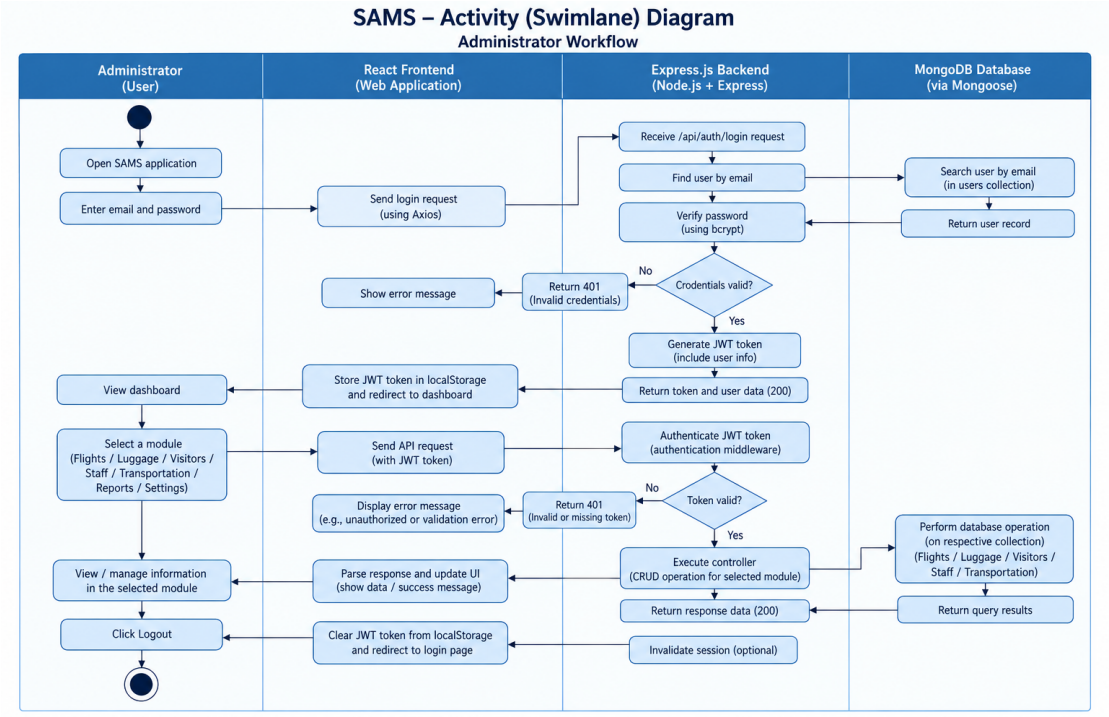

Figure 2: Activity Diagram

The authentication process begins when the user enters valid login credentials.

The React frontend sends the credentials to the backend through Axios. The backend searches for the user, verifies the password using bcrypt, and generates a JWT token after successful authentication.

The frontend stores the token and attaches it to subsequent protected API requests.

For protected operations, the backend first verifies the JWT using authentication middleware. The role middleware then checks whether the authenticated user's role is authorized for the requested operation.

The role middleware does not query MongoDB. It uses the authenticated user information available after JWT verification.

Authorized requests are then forwarded to the appropriate controller, which interacts with MongoDB through the corresponding Mongoose model.

2.3 Data Flow Diagram – Level 0

The Level 0 DFD represents SAMS as a centralized system interacting with the external users and exchanging operational information.

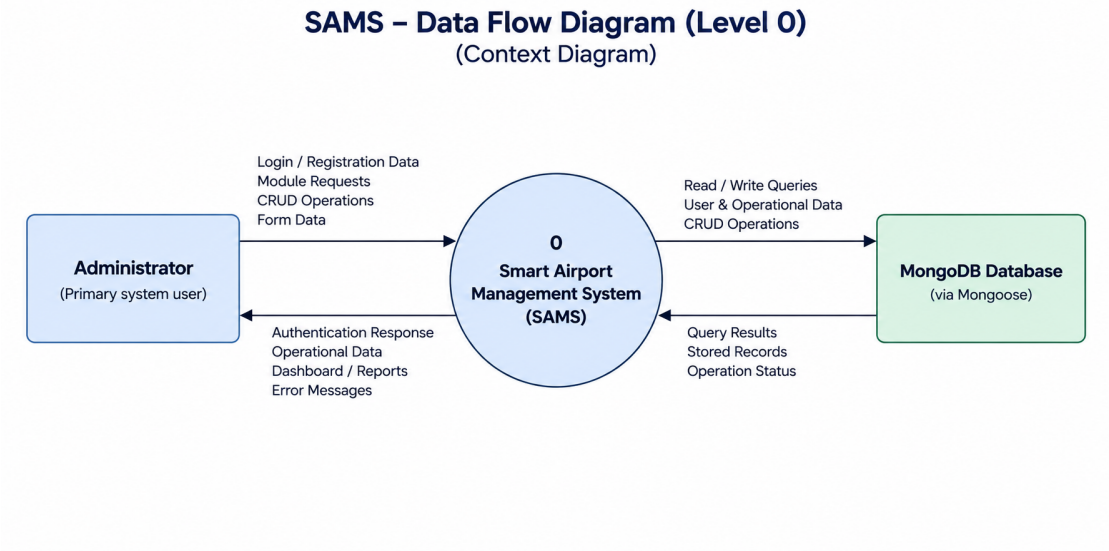

Figure 3: Data Flow Diagram Level 0

The major external actors are Administrator, Staff, Airline Staff, Passenger, Visitor, and Transportation Operator.

MongoDB is treated as an internal data store rather than an external entity at the context level.

2.4 Data Flow Diagram – Level 1

The Level 1 DFD decomposes the SAMS system into its major functional processes.

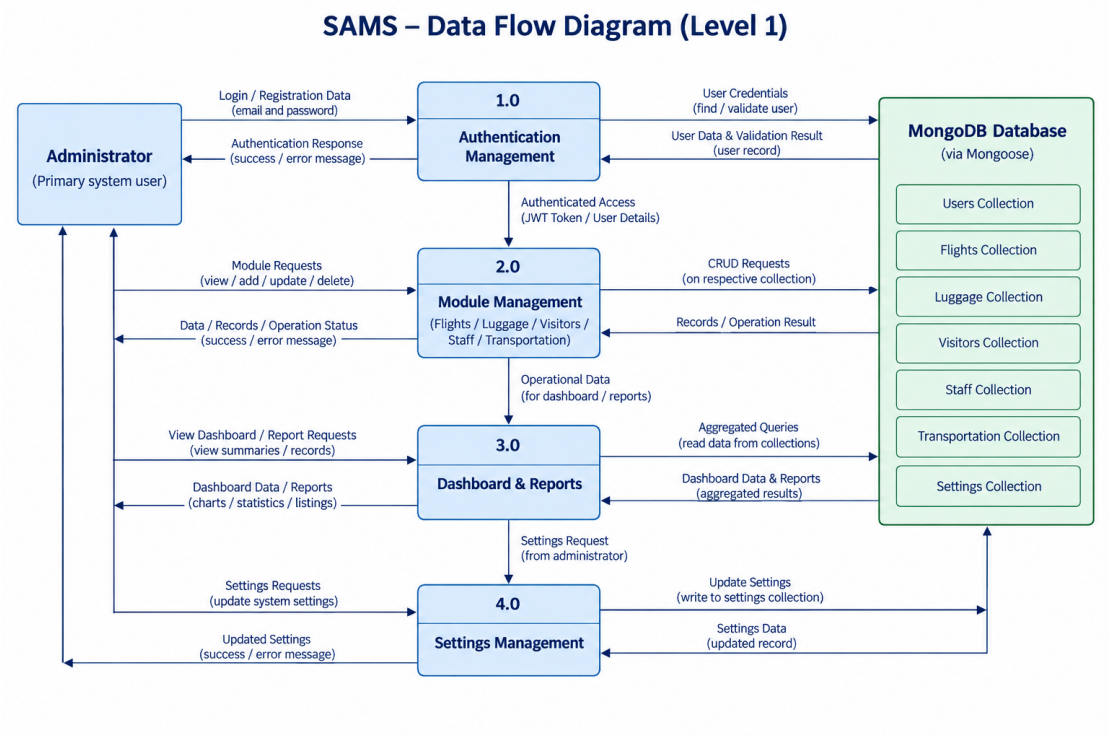

Figure 4: Data Flow Diagram Level 1

The main processes include authentication, flight management, luggage management, visitor management, staff management, transportation management, and dashboard/reporting functions.

The corresponding data is stored in MongoDB collections through the backend API.

2.5 Software Requirements Specification

2.5.1 Introduction

The Software Requirements Specification defines the functional and non-functional requirements of the Smart Airport Management System.

2.5.2 Overall Description

SAMS is a web-based airport management prototype designed to centralize operational information.

The frontend provides the user interface, while the backend provides REST APIs and communicates with MongoDB.

2.5.3 User Interfaces

The frontend provides interfaces for:

- Login
- Dashboard
- Flights
- Luggage
- Visitors
- Staff
- Transportation
- Reports
- Settings

2.5.4 Functional Requirements

Table II: Functional Requirements

ID	Requirement
FR-01	The system shall authenticate users using registered credentials.
FR-02	The system shall generate a JWT after successful authentication.
FR-03	The system shall protect API routes using JWT verification.
FR-04	The system shall apply role-based authorization to protected API operations.
FR-05	The system shall allow authorized users to manage flight information.
FR-06	The system shall allow authorized users to manage luggage information.
FR-07	The system shall allow authorized users to manage visitor information.
FR-08	The system shall allow authorized users to manage staff information.
FR-09	The system shall allow authorized users to manage transportation information.
FR-10	The system shall provide a centralized dashboard view.

2.5.5 Non-Functional Requirements

Table III: Non-Functional Requirements

Category	Requirement
Security	Authentication and authorization shall be implemented for protected API access.
Performance	API requests should be processed within reasonable response times under normal prototype usage.
Usability	The frontend should provide clear navigation and understandable management interfaces.
Maintainability	Frontend and backend modules should remain separated and modular.
Scalability	Additional modules and role-specific interfaces should be extendable in future versions.
Reliability	Database operations should return appropriate success or error responses.

2.5.6 Other Requirements

The application requires Node.js, MongoDB, and the required frontend and backend dependencies.

3. Design Phase

3.1 System Architecture

SAMS follows a three-layer architecture consisting of:

1. Presentation layer
2. Application/API layer
3. Data layer

The React frontend forms the presentation layer. The Express.js backend forms the application layer, and MongoDB forms the data layer.

3.2 Backend Architecture

The backend is organized into configuration, routes, controllers, middleware, and models.

- **Configuration:** MongoDB connection configuration.
- **Routes:** Define API endpoints.
- **Controllers:** Process application operations.
- **Middleware:** Authentication and role-based authorization.
- **Models:** Define MongoDB data structures through Mongoose.

3.3 Frontend Architecture

The frontend is implemented using React and follows a component-based structure.

The main frontend responsibilities include:

- User authentication interface.
- Protected navigation.
- API communication using Axios.
- CRUD interfaces for operational modules.
- Dashboard presentation.
- Reports and settings views.

3.4 Class Diagram

The class diagram represents the major database entities used by the application.

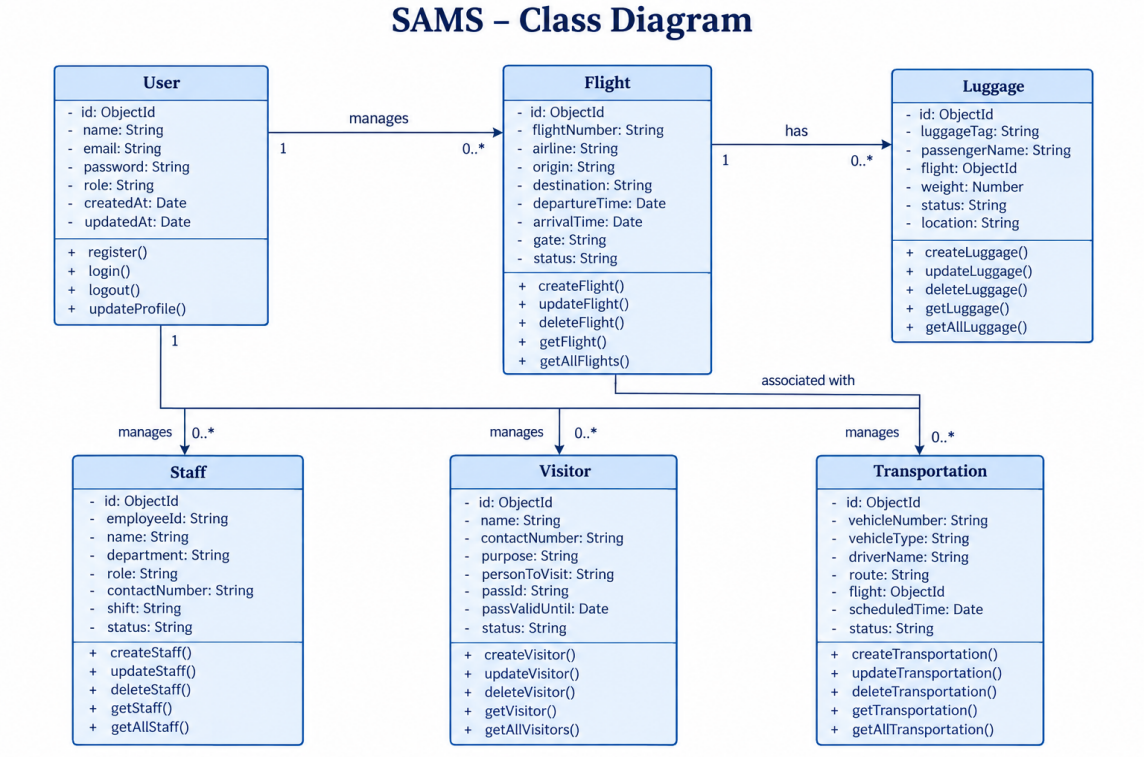

Figure 5: Class Diagram

The primary entities are User, Flight, Luggage, Staff, Transportation, and Visitor.

3.5 Entity Relationships

The database design uses references between related entities.

For example, luggage records can reference a flight, while transportation records can also reference a flight.

The relationships are implemented using MongoDB ObjectId references through Mongoose.

3.6 Sequence Diagram

The sequence diagram represents the interaction between the user, frontend, backend, authentication middleware, controller, and database.

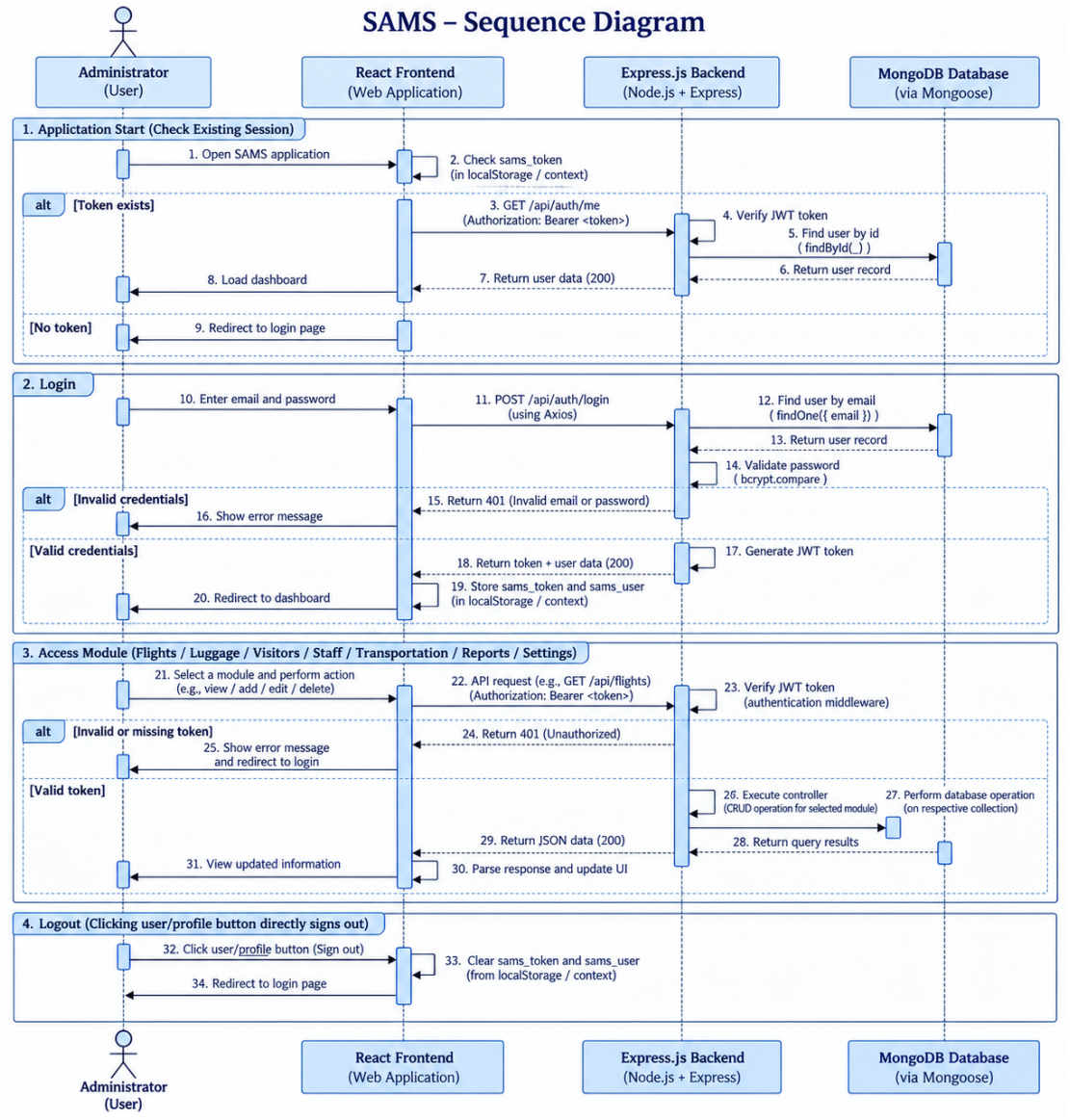

Figure 6: Sequence Diagram

3.7 Database Design

The major MongoDB collections correspond to the following Mongoose models:

- User
- Flight
- Luggage
- Staff
- Transportation
- Visitor

Each schema defines the fields, data types, constraints, and enumerated values required for the respective operational entity.

3.8 Security Design

Authentication

Authentication is performed during login.

The backend verifies the user's credentials and generates a JWT after successful authentication.

Protected API Access

Authentication and authorization are implemented at the backend API layer.

The backend first verifies the JWT token using authentication middleware. After successful authentication, role-based middleware checks whether the authenticated user's role is permitted to access the requested operation.

In the current demonstration, the Administrator is the primary authenticated user and therefore has access to the administrative management operations permitted by the backend.

The JWT is generated after successful login and is reused for subsequent protected requests. The backend automatically verifies the token on each protected request, so the administrator does not need to log in again for every operation.

Password Security

Passwords are handled using bcryptjs for password hashing and verification rather than storing plaintext passwords.

4. Implementation Phase

4.1 Frontend Implementation

The frontend is implemented using React.js and Vite.

The main interface modules include:

- Login
- Dashboard
- Flights
- Luggage
- Visitors
- Staff
- Transportation
- Reports
- Settings

Protected frontend routes prevent unauthenticated users from accessing the main management interface.

4.2 Backend Implementation

The backend is implemented using Node.js and Express.js.

The server exposes REST API endpoints under the `/api` path.

Table IV: Backend API Modules

Module	**API Base Path**
Authentication	/api/auth
Flights	/api/flights
Luggage	/api/luggage
Visitors	/api/visitors
Staff	/api/staff
Transportation	/api/transportation

4.3 Authentication Implementation

The authentication flow consists of the following steps:

1. The user enters an email and password.
2. The frontend sends the credentials to the login API.
3. The backend retrieves the corresponding user.
4. The password is verified using bcryptjs.
5. A JWT is generated after successful verification.
6. The frontend stores the JWT for the authenticated session.
7. Axios automatically attaches the token to subsequent protected API requests.
8. The backend verifies the token before processing protected requests.

4.4 Flight Management

The flight module provides API operations for managing flight information.

Flight records contain information such as flight number, airline, origin, destination, departure time, arrival time, gate, and status.

4.5 Luggage Management

The luggage module manages luggage records associated with passengers and flights.

The backend supports operations for creating, retrieving, updating, and deleting luggage records according to the permissions assigned to the authenticated role.

4.6 Visitor Management

The visitor module manages visitor registration information including visitor name, contact number, purpose, person to visit, pass information, validity period, and status.

4.7 Staff Management

The staff module manages airport staff information including employee ID, name, department, role, contact number, shift, and status.

4.8 Transportation Management

The transportation module manages transportation records including vehicle number, vehicle type, driver, route, scheduled time, related flight, and transportation status.

4.9 Dashboard

The dashboard provides a consolidated view of available operational information.

The dashboard uses the application's API communication layer to obtain available operational information and presents it in a consolidated management view.

Where complete operational data is not available, the current prototype may use predefined demonstration values for presentation purposes.

4.10 Reports

The reports section provides a consolidated view of selected system and operational information.

The current implementation is intended as a prototype reporting interface and can be extended with more advanced database-derived analytics in future versions.

4.11 Settings

The settings interface provides basic system and user-related information such as the authenticated role, API configuration, and interface preferences.

5. Results and Evaluation

5.1 Testing Methodology

Testing was performed by checking the major application workflows and API operations. The testing focused on:

- Login and authentication.
- Protected API access.
- Role-based authorization.
- CRUD operations.
- Frontend-backend communication.
- Database interaction.
- Navigation between application modules.

5.2 Representative System Test Cases

Table V: Representative System Test Cases

ID	Test Case	Expected Result	Status
TC-01	Login with valid credentials	JWT generated and user authenticated	Pass
TC-02	Login with invalid credentials	Authentication rejected	Pass
TC-03	Access protected API without token	Request rejected	Pass
TC-04	Access authorized API with valid token	Request processed	Pass
TC-05	Unauthorized role accesses restricted operation	Request rejected	Pass
TC-06	Retrieve flight records	Flight data returned	Pass
TC-07	Retrieve visitor records	Visitor data returned	Pass
TC-08	Retrieve staff records	Staff data returned	Pass
TC-09	Retrieve transportation records	Transportation data returned	Pass
TC-10	Logout/session removal	Stored authentication information removed	Pass

5.3 Results

The implemented prototype successfully establishes communication between the React frontend and Express.js backend.

The backend provides modular APIs for authentication, flights, luggage, visitors, staff, and transportation.

MongoDB is used for persistent storage through Mongoose models.

The authentication system generates JWT tokens after successful login and verifies those tokens on subsequent protected API requests.

5.4 Administrator Workflow

The current prototype demonstrates the following centralized workflow:

1. Administrator opens the application.

2. Administrator logs in using valid credentials.
3. Backend authenticates the administrator and returns a JWT.
4. Frontend stores the authentication information.
5. Administrator accesses the centralized dashboard.
6. Administrator navigates between operational modules.
7. Requests are sent to the backend with the JWT attached automatically.
8. Backend verifies the JWT and applies role-based authorization.
9. Authorized operations are performed through the appropriate controller and database model.

5.5 Limitations

The current implementation is a prototype and has several limitations:

- The demonstrated frontend workflow is primarily administrator-centric.
- Separate dedicated interfaces for all backend roles are not currently implemented.
- Some dashboard and reporting values may use predefined demonstration data.
- Advanced analytics and real-time airport integrations are outside the current prototype scope.
- Production deployment, monitoring, and large-scale performance testing are not included.

5.6 Security and Privacy

The application uses JWT-based authentication, bcrypt password hashing, and role-based authorization middleware.

Protected API routes require a valid JWT.

The role middleware ensures that authenticated users can only access operations permitted for their assigned role.

The current prototype should undergo additional security hardening before production deployment, including secure secret management, production TLS configuration, stronger validation, rate limiting, logging, and comprehensive security testing.

5.7 Analysis

The modular architecture separates the frontend presentation layer from backend application logic and database operations.

This separation improves maintainability because changes to a particular layer can generally be made without restructuring the entire application.

5.8 Centralization

A major benefit of SAMS is the centralized management interface.

Instead of requiring separate applications for different airport information categories, the prototype provides a single interface through which the administrator can navigate between flights, luggage, visitors, staff, transportation, dashboard, reports, and settings.

5.9 Trade-Offs

The current prototype prioritizes modularity and centralized management over advanced real-time integration.

The architecture provides a foundation for future extensions while keeping the current implementation manageable for a prototype.

5.10 Maintainability

The separation of routes, controllers, middleware, models, and frontend components provides a structured codebase.

This organization makes it easier to add new modules or modify existing functionality in future iterations.

6. Detailed System Description

6.1 System Workflow and Component Interaction

SAMS follows a clear request-response workflow between the frontend, backend, and database. The React frontend is responsible for presenting the application interface and collecting user actions. When the administrator performs an operation, the frontend communicates with the Express.js backend through REST APIs using Axios. The backend receives the request, performs authentication and authorization checks where required, executes the relevant controller logic, and communicates with MongoDB through Mongoose.

The separation between these layers provides a structured flow of information. The frontend does not directly access the database. Instead, database operations are performed by the backend. This provides a central point for authentication, authorization, validation, and application logic. It also makes the system easier to maintain because the user interface and data-management logic remain separated.

The authentication workflow is completed before protected operational requests are processed. During login, the submitted credentials are sent to the authentication API. The backend retrieves the corresponding user record and verifies the password. If verification succeeds, a JWT is generated and returned to the frontend. The token is then stored as part of the authenticated session and is attached to subsequent protected requests.

For a protected module request, the backend verifies the JWT before allowing the controller to execute the requested operation. The authenticated user's role is then considered by the role-based middleware. If the request is authorized, the appropriate controller performs the required CRUD operation using the corresponding Mongoose model. The database response is returned through the backend and then presented by the React interface.

This workflow establishes a consistent pattern across the major modules. Whether the administrator is viewing flights, managing luggage, updating visitor information, managing staff, or handling transportation records, the same layered communication approach is followed.

6.2 Authentication and Session Management

Authentication is one of the central security mechanisms in SAMS. The purpose of authentication is to confirm that a user is using valid registered credentials before access to the management interface and protected API operations is provided. The current prototype demonstrates this workflow primarily through the administrator account.

The login process starts when the user enters an email address and password in the frontend. Axios sends these credentials to the backend login endpoint. The backend searches the Users collection for the corresponding account. The stored password is not treated as a plaintext value; bcryptjs is used to verify the supplied password against the stored password representation.

When the credentials are invalid, the backend rejects the request and returns an authentication error. The frontend can then display an error message to the user. When the credentials are valid, the backend generates a JWT. The response contains the authentication token together with user information required by the frontend. The frontend stores the authentication information for the current session.

The token is important because the user does not need to authenticate again for every operation. Instead, the token is attached to subsequent protected API requests. The backend verifies the token through authentication middleware. A request without a valid token is rejected before the protected controller is executed.

The application also supports checking the authenticated user through the authentication API. This allows the frontend to restore or validate an existing session when the application is opened again. If a valid stored token is available, the frontend can request the authenticated user's information and continue to the dashboard. If there is no usable session, the application redirects the user toward authentication.

Logout is handled at the frontend session level in the current prototype. The stored authentication information is cleared and the user is redirected to the login page. Clearing the stored token prevents it from being reused by the frontend for later protected requests.

Overall, the authentication design combines password verification, JWT-based sessions, protected frontend navigation, and backend token verification. This creates a consistent authentication mechanism across the application.

6.3 Authorization and Protected API Design

Authentication and authorization perform different functions in SAMS. Authentication establishes that a request is associated with an authenticated user, while authorization determines whether that authenticated user's role is permitted to perform a particular operation.

The backend contains role-based authorization middleware. After the JWT has been successfully verified, the middleware can use the authenticated user's information to determine the assigned role. The role middleware does not need to perform another database query merely to determine the role when that information is already available from the authenticated request context.

The current prototype is administrator-centric from the frontend perspective. The Administrator is the primary user demonstrated in the application, and the administrative workflow provides access to the operational management modules. At the backend level, the project also defines additional roles such as Staff, Airline Staff, Passenger, Visitor, and Transportation Operator. These roles provide a basis for extending the system with more specialized permissions and interfaces.

A protected request therefore follows a sequence of checks. First, the frontend sends the request with the JWT attached. Second, the authentication middleware verifies the token. Third, role middleware checks the authorization condition for the requested operation. Only after these checks succeed is the request passed to the relevant controller.

If the token is missing, invalid, or expired, the request can be rejected with an unauthorized response. If the token is valid but the role is not permitted for the requested operation, the backend can return a forbidden response. These two responses represent different security conditions and help separate authentication failures from authorization failures.

This design is useful because authorization is enforced at the backend rather than relying only on frontend navigation. Even if a user attempts to call an API directly, the protected backend route still performs the authentication and authorization checks. This provides a stronger security boundary than hiding or disabling interface elements alone.

The architecture is also extensible. New roles or permissions can be introduced in future versions without requiring the complete application structure to be redesigned. Dedicated role-specific dashboards can similarly be developed while continuing to use the same authentication and authorization foundation.

6.4 Operational Module Design

The operational part of SAMS is divided into multiple modules so that different categories of airport information can be managed independently. The major modules represented in the current design are Flights, Luggage, Visitors, Staff, and Transportation.

The Flight module manages flight information. A flight record contains fields such as flight number, airline, origin, destination, departure time, arrival time, gate, and status. The backend exposes a dedicated API base path for flight operations. CRUD operations allow the application to create, retrieve, update, and delete flight records according to the permissions of the authenticated user.

The Luggage module manages luggage records associated with passengers and flights. Information such as luggage tag, passenger name, related flight, weight, status, and location can be represented in the corresponding data model. This module allows luggage information to be maintained independently while still supporting relationships with flight records.

The Visitor module manages visitor information. The represented fields include visitor name, contact number, purpose, person to visit, pass information, validity period, and status. These records can be created, retrieved, updated, and deleted through the backend API.

The Staff module manages airport staff information. Staff records contain information such as employee ID, name, department, role, contact number, shift, and status. The module provides the same general CRUD pattern used by the other operational modules.

The Transportation module manages airport transportation records. The model can contain vehicle number, vehicle type, driver information, route, scheduled time, related flight, and transportation status. These records can be maintained through the transportation API.

Although the modules represent different operational categories, they follow a common architectural pattern. The React frontend provides the management interface, Axios sends the API request, Express routes the request to the appropriate controller, and the controller communicates with MongoDB through Mongoose. This common structure reduces duplication in the overall design and makes additional modules easier to incorporate.

The centralized module approach is one of the main design characteristics of SAMS. Instead of requiring separate interfaces for every information category, the administrator can move between the modules from one web application.

6.5 Database and Data Management

MongoDB is used as the persistent database for SAMS, while Mongoose provides the object data modeling layer between the Express.js backend and MongoDB. The database design is divided into collections corresponding to the major application entities.

The main collections represented by the system are Users, Flights, Luggage, Visitors, Staff, Transportation, and Settings. Each collection stores records associated with its respective application function. Mongoose models define the expected structure of these records and provide a consistent way for controllers to interact with the database.

The User model supports authentication and user-related information. User records include information such as name, email, password representation, role, creation time, and update time. The role information is relevant to the authorization mechanism used by protected API routes.

The Flight model stores information required to represent airport flight operations. Luggage and Transportation records can contain references to flight records. These references use MongoDB ObjectId values through Mongoose, allowing related records to be connected without duplicating the complete flight record in every related document.

CRUD operations are performed through the backend controllers. A create operation adds a new record, a read operation retrieves existing records, an update operation modifies an existing record, and a delete operation removes a record when permitted. The frontend does not directly perform these database operations; it communicates with the backend APIs instead.

Using Mongoose also provides a structured location for schema definitions, field types, and validation rules. This helps maintain consistency in the stored information and reduces the possibility of arbitrary data structures being inserted into the database.

The database architecture supports the centralized design of the application. The dashboard and reporting components can obtain information from the available operational records, while individual modules can focus on their own collection and controller logic.

The database design can also be expanded in future versions. Additional collections, relationships, indexes, and validation rules can be introduced as the functional requirements of the system grow. This is particularly relevant if the prototype is extended toward real-time airport data, more detailed analytics, or dedicated interfaces for additional user roles.

6.6 Frontend and Backend Communication

The frontend-backend communication model is based on REST APIs. React provides the presentation layer, while Express.js provides the backend API layer. Axios acts as the communication mechanism used by the frontend to send HTTP requests to the backend.

The API routes are organized by application function. Authentication operations use the `/api/auth` path, while the major operational modules use paths such as `/api/flights`, `/api/luggage`, `/api/visitors`, `/api/staff`, and `/api/transportation`. This organization makes the backend interface easier to understand and maintain.

A typical protected request begins with an action in the React interface. For example, when the administrator requests flight information, the frontend prepares an API request and includes the JWT in the authorization information. The Express.js backend receives the request and passes it through the applicable middleware.

Authentication middleware verifies the token. If the token is invalid, the request does not continue to the controller. If authentication succeeds, role authorization is applied according to the route requirements. Once the request is authorized, the controller performs the relevant operation through the Mongoose model.

The backend then returns an HTTP response containing the requested data, a success response, or an appropriate error. The frontend receives the response through Axios and updates the displayed interface. This request-response cycle is repeated for different CRUD operations and modules.

This architecture has several practical advantages. First, it keeps database credentials and database logic on the backend instead of exposing them to the browser. Second, authentication and authorization can be applied consistently to protected routes. Third, the frontend can be redesigned independently as long as the API contract remains compatible.

The communication layer also provides a clear boundary for debugging. If an operation fails, the problem can be investigated at the frontend request, backend route, middleware, controller, or database level. This separation is useful during development and testing because each layer has a defined responsibility.

The current prototype therefore demonstrates a conventional full-stack web application flow: user interaction in React, HTTP communication through Axios, request processing in Express.js, database interaction through Mongoose, and persistent storage in MongoDB.

6.7 Dashboard, Reports, Settings and User Experience

The dashboard provides a consolidated view of operational information available through the application. Its purpose is to give the administrator a central location from which important information can be viewed without opening each management module independently.

The frontend contains separate navigation areas for Flights, Luggage, Visitors, Staff, Transportation, Reports, and Settings. This structure reflects the centralized nature of the system. After authentication, the administrator can navigate between these areas using the application interface.

The dashboard can use available API data to present operational information in a consolidated form. The project also includes reporting and visualization support through the frontend technology stack. Where complete operational information is not available in the current prototype, predefined demonstration values may be used for presentation purposes. This distinction is important because the current system is a prototype rather than a production airport information platform.

The Reports section provides a consolidated view of selected system and operational information. It is intended as a prototype reporting interface and can be extended with more advanced analytics in future versions. Future reporting improvements can include fully database-derived metrics and additional visualizations.

The Settings section provides basic system and user-related information. The current interface can display information such as the authenticated role, API configuration, and interface preferences. Settings therefore act as a supporting part of the application rather than a separate operational data-management module.

The user experience is supported by protected navigation. Unauthenticated users should not be able to access the main management interface through normal protected routes. After authentication, the user can navigate through the available modules while the JWT is automatically attached to protected API requests.

Logout removes the stored authentication information from the frontend session and returns the user to the login page. This provides a clear end to the current authenticated session.

Overall, the dashboard, reports, and settings features complement the CRUD modules. The operational modules provide detailed management functions, while the dashboard and reporting views provide a higher-level representation of the available information. This combination supports the project's goal of providing a centralized airport management interface.

6.8 Testing, Limitations and Future Extensions

Testing of the current prototype focuses on the major workflows and the communication between the application layers. The representative test cases cover valid and invalid login attempts, protected API access, authorization behavior, retrieval of operational records, and session removal during logout.

A valid login should result in successful authentication and JWT generation, while invalid credentials should be rejected. A protected API request without a valid token should also be rejected. For an authenticated and authorized request, the appropriate controller should execute and return the requested information. These tests help verify the main security and functional paths of the prototype.

The current implementation has defined limitations. The demonstrated frontend workflow is primarily administrator-centric, even though additional roles are represented in the backend architecture. Separate dedicated interfaces for every defined role are not currently implemented. Some dashboard or reporting values may also use predefined demonstration data rather than complete live analytics.

The prototype is not intended to represent a complete production airport infrastructure. Real-time flight integrations, real-time luggage tracking, airport mapping, advanced analytics, large-scale performance testing, production deployment, and continuous monitoring remain outside the current implementation scope.

Security is another area for future hardening. The current system already uses JWT authentication, bcrypt password handling, and role-based authorization middleware. Before production deployment, additional measures such as secure secret management, TLS configuration, stronger validation, rate limiting, logging, and comprehensive security testing should be considered.

The architecture nevertheless provides a foundation for further development. Dedicated dashboards can be created for Staff, Airline Staff, Passenger, Visitor, and Transportation Operator. Additional airport modules can be introduced using the same frontend, backend, and database separation. Real-time data sources can later be integrated with the existing API layer.

Future development can also improve reporting, notifications, operational analytics, airport map visualization, and cloud deployment. These extensions can be implemented incrementally without changing the basic three-layer architecture.

The testing and limitation analysis therefore shows both the current capabilities and the boundaries of the prototype. The system demonstrates the core centralized management concept while leaving clear opportunities for future development.

7. Conclusion

7.1 Conclusion

The Smart Airport Management System provides a centralized web-based platform for organizing and managing airport-related operational information.

The prototype combines React.js, Express.js, Node.js, MongoDB, Mongoose, JWT authentication, bcrypt password security, and role-based authorization into a modular architecture.

The current demonstration focuses on the administrator workflow, while the backend provides role definitions and API-level authorization for additional users.

The system successfully demonstrates the core concept of centralized airport information management and provides a foundation for further development.

7.2 Findings

The implementation demonstrates that a modular full-stack architecture can be used to consolidate multiple airport management functions into a single application.

The separation of frontend, backend, middleware, controllers, models, and database components supports maintainability and future expansion.

7.3 Future Work

Future development can include:

- Dedicated dashboards for Staff, Airline Staff, Passenger, Visitor, and Transportation Operator.
- Real-time flight status integration.
- Real-time luggage tracking.

- Airport map and gate visualization.
- Advanced operational analytics.
- Notifications and alerts.
- Improved reporting with fully database-derived analytics.
- Stronger production-level security and monitoring.
- Deployment to a cloud environment.

References

1. React Documentation, React.js. https://react.dev/
2. Express.js Documentation. https://expressjs.com/
3. Node.js Documentation. https://nodejs.org/
4. MongoDB Documentation. https://www.mongodb.com/docs/
5. Mongoose Documentation. https://mongoosejs.com/docs/
6. Axios Documentation. https://axios-http.com/
7. JSON Web Tokens. https://jwt.io/
8. Vite Documentation. https://vite.dev/
9. Git Documentation. https://git-scm.com/doc
10. GitHub Documentation. https://docs.github.com/

A. Authentication Endpoints

Table VI: Authentication API Endpoints

Method	**Endpoint**	**Purpose**
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate a user and generate JWT
GET	/api/auth/me	Retrieve authenticated user information

B. Operational API Endpoints

Table VII: Operational API Endpoints

Module	**Endpoint**	**Operations**
Flights	/api/flights	Create, Read, Update, Delete
Luggage	/api/luggage	Create, Read, Update, Delete
Visitors	/api/visitors	Create, Read, Update, Delete
Staff	/api/staff	Create, Read, Update, Delete
Transportation	/api/transportation	Create, Read, Update, Delete

C. Project Structure

The major project components are organized as follows:

```
SAMS/
|
+-- backend/
\|   +-- src/
\|       +-- config/
\|       +-- controllers/
\|       +-- middleware/
\|       +-- models/
\|       +-- routes/
\|       +-- server.js
|
+-- frontend/
\|   +-- src/
\|       +-- components/
\|       +-- context/
\|       +-- pages/
\|       +-- lib/
\|       +-- App.jsx
|
+-- docs/
+-- assets/
+-- project-proposal/
+-- project-report-final/
+-- project-report-prototype-stage/
+-- journals/
```

D. Deployment and Execution

Backend

The backend can be started using the following commands:

```
cd backend
npm install
npm run dev
```

The backend requires environment variables such as:

```
MONGO\_URI=your\_mongodb\_connection\_string
JWT\_SECRET=your\_secret\_key
PORT=5000
```

Frontend

The frontend can be started using:

```
cd frontend
npm install
npm run dev
```

The frontend communicates with the backend through the configured API URL.